# Explicación de la práctica: HTML y CSS sin flex ni grid

Esta guía explica, parte por parte, qué hace cada cosa del `index.html` y del `estilos.css`, por qué se ha usado y cómo puedes reutilizarlo tú.

## 1. Cómo se reparten el trabajo HTML y CSS

- **HTML** define **qué hay** en la página: una cabecera, un menú, una barra lateral, un texto, un pie... Es la estructura (el "esqueleto").
- **CSS** define **cómo se ve y dónde se coloca**: colores, tamaños, bordes, posición de cada caja.

Los dos archivos se conectan con esta línea del HTML:

```
<link rel="stylesheet" href="estilos.css">
```

- `rel="stylesheet"`: le dice al navegador que el archivo enlazado es una hoja de estilos.
- `href="estilos.css"`: la ruta del archivo. Como está en la misma carpeta que el HTML, basta con el nombre.

## 2. El HTML explicado

### 2.1. La parte de arriba (la "cabecera técnica")

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Práctica de maquetación</title>
  <link rel="stylesheet" href="estilos.css">
</head>
```

| Línea | Qué hace | Por qué |
|---|---|---|
| `<!DOCTYPE html>` | Indica que el documento es HTML5. | Sin ella, el navegador entra en un "modo antiguo" (quirks mode) y el CSS puede comportarse raro. |
| `<html lang="es">` | Es la etiqueta raíz; todo va dentro. `lang="es"` dice que el idioma es español. | Ayuda a lectores de pantalla, traductores y buscadores. |
| `<head>` | Zona de información que **no se ve** en la página. | Aquí van configuración, título y enlaces a CSS. |

| Línea | Qué hace | Por qué |
|---|---|---|
| `<meta charset="UTF-8">` | Codificación de caracteres. | Para que se vean bien la ñ, las tildes (párrafo, García, Imágenes). |
| `<meta name="viewport" ...>` | Le dice al móvil que use el ancho real de la pantalla. | Sin esto, en móvil la página se ve diminuta. |
| `<title>` | Texto que sale en la pestaña del navegador. | Obligatorio y útil para buscadores. |
| `<link ...>` | Enlaza el CSS. | Explicado arriba. |

### 2.2. El `<body>`: lo que se ve

Todo lo visible va dentro de `<body>`. La página tiene cuatro bloques, uno debajo de otro:

```
header  (cabecera)
nav     (menú horizontal)
div     (cuerpo: lateral + contenido)
footer  (pie)
```

### 2.3. Etiquetas semánticas

En lugar de usar `<div>` para todo, se usan etiquetas con **significado** (semánticas):

| Etiqueta | Significado | Dónde se usa |
|---|---|---|
| `<header>` | Cabecera de la página | Caja con "Cabecera" y "Buscar" |
| `<nav>` | Zona de navegación (menú) | Barra de Enlace1…Enlace4 |
| `<aside>` | Contenido secundario/lateral | Barra lateral izquierda |
| `<section>` | Una sección de contenido | Caja verde central |
| `<footer>` | Pie de página | Caja rosa |
| `<div>` | Caja genérica **sin** significado | `.cuerpo`, cajas cian, cajas del pie |

Visualmente un `<header>` y un `<div>` son iguales: los dos son bloques. La diferencia es que las semánticas hacen el código más legible y ayudan a la accesibilidad y al SEO. Usa `<div>` solo cuando ninguna otra etiqueta encaje.

### 2.4. La cabecera

```
<header class="cabecera">
  <h1>Cabecera</h1>
  <a class="boton-buscar" href="#">Buscar</a>
</header>
```

- `<h1>`: título principal. Debería haber **un solo h1** por página.

- `<a href="#">`: un enlace. `#` significa "no va a ninguna parte" (se usa como enlace provisional en maquetas).
- `class="..."`: un nombre que ponemos nosotros para poder seleccionar ese elemento desde el CSS.

### 2.5. Los menús (listas)

```
<nav class="menu">
  <ul>
    <li><a href="#">Enlace1</a></li>
    ...
  </ul>
</nav>
```

- `<ul>`: lista desordenada (por defecto con puntitos).
- `<li>`: cada elemento de la lista.
- `<a>`: el enlace dentro de cada elemento.

Un menú es, en el fondo, **una lista de enlaces**; por eso se escribe así. Luego con CSS quitamos los puntitos y los colocamos en fila. La barra lateral usa exactamente la misma estructura, solo que dentro de un `<aside>`.

### 2.6. El cuerpo

```
<div class="cuerpo">
  <aside class="lateral"> ... </aside>
  <section class="contenido">
    <div class="imagen-central"></div>
    <div class="texto-central">
      <h2>Textos centrales</h2>
      <p>Este es el párrafo...</p>
    </div>
  </section>
</div>
```

- `.cuerpo` es un **contenedor**: agrupa la lateral y el contenido para que vayan en la misma "fila".
- `.imagen-central` está vacía: en la maqueta es un hueco reservado para una imagen.
- `<h2>`: subtítulo (un nivel por debajo de h1).
- `<p>`: párrafo.

### 2.7. El pie y las clases múltiples

```
<div class="caja caja-direccion">
```

Un elemento puede tener **varias clases separadas por espacios**. Aquí:

- `caja` → estilos **comunes** a las tres cajas (fondo azul, texto blanco, borde...).
- `caja-direccion` → estilos **propios** de esa caja (su ancho, alto y margen).

Así no repites el mismo código tres veces. Es un truco muy útil.

### 2.8. Comentarios

```
<!-- Cabecera -->
```

No se muestran en la página. Sirven para organizar y explicar tu código.

## 3. Conceptos básicos de CSS antes de empezar

### 3.1. Cómo es una regla CSS

```
selector {
  propiedad: valor;
}
```

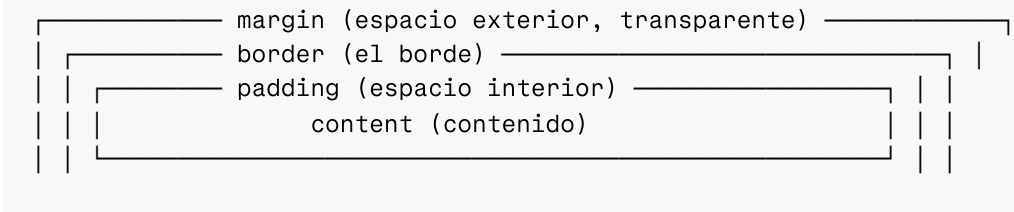
### 3.2. Los selectores usados

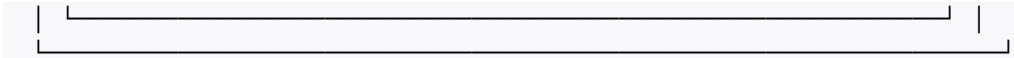
| Selector | Qué selecciona | Ejemplo |
|---|---|---|
| `*` | **Todos** los elementos | Reset |
| `body`, `a`, `ul` | Todas las etiquetas de ese tipo | `a { color: blue; }` |
| `.nombre` | Elementos con esa clase | `.cabecera` |
| `.menu ul li` | Los `li` que están dentro de un `ul` que está dentro de `.menu` (selector descendiente) | Enlaces del menú |
| `.cabecera h1` | El `h1` que está dentro de `.cabecera` | El título azul |

El selector descendiente es clave: `.menu ul li` y `.lateral ul li` son ambos `li`, pero gracias al contenedor les damos estilos **distintos** (uno en fila, otro en columna).

### 3.3. El modelo de caja

Todo elemento es una caja con cuatro capas, de dentro hacia fuera:





- `margin`: separa la caja de **las demás**.
- `padding`: separa el contenido **del borde**.
- `border`: la línea de alrededor.

Formas de escribir margin/padding:

```
margin: 20px;                /* los 4 lados */
margin: 16px 0 14px;         /* arriba | izquierda y derecha | abajo */
margin: 15px 0 0 2%;         /* arriba | derecha | abajo | izquierda (sentido del re
margin-top: 18px;            /* solo un lado */
```

### 3.4. Elementos de bloque y en línea

- **Bloque** (`div`, `p`, `h1`, `ul`, `li`, `header`...): ocupan todo el ancho disponible y empiezan en una línea nueva. Por eso, sin CSS, todo sale **uno debajo de otro**.
- **En línea** (`a`, `span`...): ocupan solo lo que mide su contenido y van seguidos dentro del texto.

Para poner cajas de bloque **una al lado de otra** sin flex ni grid necesitamos **float** o **position**.

### 3.5. Unidades

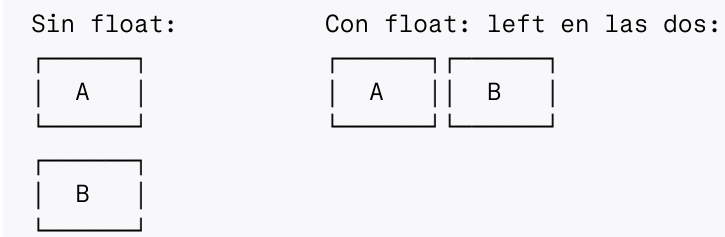
| Unidad | Significado | Cuándo usarla |
|---|---|---|
| `px` | Píxeles, tamaño fijo | Alturas, bordes, márgenes pequeños, tamaño de letra |
| `%` | Porcentaje del **ancho del padre** | Anchos que deben adaptarse a la pantalla |

En la última versión pasamos los anchos de `px` a `%` para que la página ocupe la pantalla entera. Las alturas se dejaron en `px` para respetar la maqueta.

## 4. Las dos herramientas de colocación: float y position

### 4.1. `float`

`float: left` / `float: right` saca un elemento del flujo normal y lo empuja a la izquierda o a la derecha de su contenedor. Los siguientes elementos flotantes se colocan **a su lado** si caben.



Si la suma de anchos supera el 100 %, la caja que no cabe **baja a la línea siguiente**. Por eso siempre hay que vigilar que los porcentajes sumen 100 % o menos.

**Problema importante de los floats:** como los elementos flotantes "salen del flujo", el contenedor padre **no los cuenta para su altura** y puede quedarse con altura 0 (se "colapsa"). Hay tres soluciones habituales:

1. Dar una **altura fija** al padre (`height: 310px`) → **es la que usamos aquí**, porque la maqueta tiene medidas fijas.
2. Poner `overflow: hidden` al padre (lo usamos en la primera versión).
3. Añadir un elemento con `clear: both` al final (técnica *clearfix*).

### 4.2. `position`

| Valor | Qué hace |
|---|---|
| `static` | El valor por defecto. El elemento sigue el flujo normal; `top`, `left`… no hacen nada. |
| `relative` | Sigue en su sitio normal, pero puedes moverlo con `top`/`left`/… respecto a donde estaría. **Además, sirve de referencia para sus hijos** `absolute`**.** |
| `absolute` | Sale del flujo y se coloca con `top`/`right`/`bottom`/`left` respecto al **ancestro más cercano que tenga position distinta de static**. Si no hay ninguno, respecto a la página. |
| `fixed` | Como absolute, pero respecto a la **ventana**: se queda fijo aunque hagas scroll (típico en menús fijos o botones de chat). |
| `sticky` | Se comporta como relative hasta que, al hacer scroll, llega a una posición (`top: 0`), y entonces se "pega" como fixed. |

En esta práctica usamos la pareja más típica: **padre** `relative` **+ hijos** `absolute` en la cabecera.

## 5. El CSS explicado bloque a bloque

### 5.1. Reset básico

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
```

- `margin: 0; padding: 0;`: los navegadores ponen márgenes por defecto a `h1`, `p`, `ul`, `body`… y cada navegador pone unos distintos. Los quitamos todos para partir de cero y controlar nosotros cada espacio.
- `box-sizing: border-box;`: hace que `width` y `height` **incluyan el padding y el borde**.
  - Sin él (`content-box`, el valor por defecto): una caja con `width: 30%` + `border: 1px` mide 30 % + 2px → las cuentas no cuadran y los floats se descolocan.

- Con él: `width: 30%` mide exactamente 30 %, tenga el borde y el padding que tenga. **Imprescindible cuando trabajas con porcentajes.**

### 5.2. Estilos generales

```
body {
  font-family: "Times New Roman", Times, serif;
  font-size: 12px;
  margin: 20px;
}
```

- `font-family`: lista de fuentes por orden de preferencia. Si no tiene la primera, prueba la segunda, y si no, cualquier fuente con remate (`serif`). Se usa Times porque la maqueta está en esa letra.
- `font-size: 12px`: tamaño base. Todos los elementos lo heredan salvo que se cambie.
- `margin: 20px`: deja 20px de aire alrededor de toda la página. Al **no** poner `width`, el body ocupa todo el ancho de la ventana (menos esos márgenes). En la versión anterior tenía `width: 1120px`, que es lo que la "limitaba".

```
a { color: blue; }
ul { list-style: none; }
```

- `color: blue` en los enlaces: el texto en azul.
- `list-style: none`: quita los puntitos de las listas.

### 5.3. Cabecera

```
.cabecera {
  position: relative;
  height: 100px;
  border: 1px solid #444;
  border-radius: 6px;
  margin-bottom: 20px;
}
```

- `position: relative`: **no mueve** la cabecera. La ponemos para que sea la **referencia** de los elementos `absolute` de dentro. Sin esto, "Buscar" se colocaría respecto a toda la página.
- `height: 100px`: altura fija, como en la maqueta.
- `border: 1px solid #444`: borde de 1px, línea continua (`solid`), color gris oscuro. Otros estilos: `dashed` (rayas), `dotted` (puntos).
- `border-radius: 6px`: redondea las esquinas.
- `margin-bottom: 20px`: separa la cabecera del menú.

**Colores:** `#444` es un color en hexadecimal (`#rrggbb` o abreviado `#rgb`). `#000` es negro, `#fff` blanco, `#444` gris oscuro. También se pueden usar nombres: `blue`, `red`, `pink`.

```
.cabecera h1 {
  position: absolute;
  top: 21px;
  left: 5%;
  width: 15%;
  height: 30px;
  line-height: 28px;
  background-color: blue;
  color: white;
  font-size: 24px;
  text-align: center;
  border: 1px solid #333;
  border-radius: 4px;
}
```

- `position: absolute; top: 21px; left: 5%;`: lo colocamos a 21px del borde superior y al 5 % del borde izquierdo de `.cabecera`.
- `width: 15%`: ocupa el 15 % del ancho de la cabecera.
- `background-color`: color de fondo. `color`: color del texto.
- `text-align: center`: centra el texto **horizontalmente**.
- **Truco para centrar en vertical:** `line-height: 28px`. Si la altura de la línea es igual a la altura de la caja, el texto queda centrado verticalmente. Es 28 y no 30 porque la caja mide 30px **incluyendo** 1px de borde arriba y 1px abajo (gracias a `border-box`): 30 − 2 = 28. Ojo: el truco solo funciona con textos de una línea.

```
.boton-buscar {
  position: absolute;
  top: 5px;
  right: 0;
  width: 10%;
  height: 16px;
  line-height: 14px;
  ...
  text-decoration: none;
}
```

- `right: 0`: lo pega al borde derecho de la cabecera. Puedes usar `left` o `right`, `top` o `bottom`, según desde dónde quieras medir.
- `text-decoration: none`: quita el subrayado que los enlaces traen por defecto, para que parezca un botón.
- `line-height: 14px`: mismo truco de centrado (16 − 2 de borde).

### 5.4. Menú horizontal

```
.menu {
  height: 67px;
  border: 1px solid #444;
  border-radius: 6px;
  margin-bottom: 21px;
}
```

La altura fija de 67px crea el espacio vacío debajo de los enlaces que aparece en la maqueta, y además evita el problema del padre colapsado por los floats.

```
.menu ul li {
  float: left;
  width: 23.5%;
  height: 14px;
  line-height: 12px;
  margin-right: 0.3%;
  border: 1px solid black;
  text-align: center;
}
```

- `float: left`: los `li`, que son bloques (uno debajo de otro), pasan a colocarse **en fila** de izquierda a derecha.
- `width: 23.5%` + `margin-right: 0.3%`: cada enlace ocupa 23,8 % → los cuatro suman 95,2 %. Sobra un hueco a la derecha, igual que en la maqueta. Si pasaras de 100 %, el cuarto bajaría a otra línea.
- `line-height: 12px`: centrado vertical (14 − 2).
- `text-align: center`: centra el texto del enlace en su celda.

### 5.5. Cuerpo

```
.cuerpo {
  height: 310px;
  margin-bottom: 20px;
}
```

Sus dos hijos (lateral y contenido) son flotantes, así que sin esta altura `.cuerpo` mediría 0 y el pie se metería por debajo de ellos. Con `height: 310px` el contenedor ocupa su sitio.

```
.lateral {
  float: left;
  width: 30%;
  height: 310px;
  border: 1px solid #444;
  border-radius: 5px;
}
```

Flota a la izquierda y ocupa el 30 %.

```
.lateral ul li {
  height: 30px;
  padding-top: 1px;
}
```

Aquí los `li` **no** flotan, así que siguen uno debajo de otro. Con `height: 30px` cada enlace ocupa 30px de alto y queda el espacio entre ellos que se ve en la maqueta.

```
.contenido {
  float: right;
  width: 67%;
  height: 310px;
  padding: 15px;
  background-color: #00ff55;
  border-radius: 5px;
}
```

- `float: right`: se pega a la derecha.
- 30 % (lateral) + 67 % (contenido) = 97 % → el 3 % restante es el **hueco entre las dos columnas**. No hace falta ningún margen: sale solo porque una va a la izquierda y otra a la derecha.
- `padding: 15px`: el borde verde que rodea las cajas cian. Gracias a `border-box`, este padding no hace la caja más grande.

```
.imagen-central {
  float: left;
  width: 41.5%;
  height: 279px;
  background-color: #00ffee;
  border-top: 1px solid #555;
  border-left: 1px solid #555;
  border-radius: 5px;
}

.texto-central {
  float: right;
  width: 56.5%;
  height: 279px;
  background-color: #00ffee;
  border-radius: 5px;
}
```

- Es **el mismo esquema** que lateral + contenido, pero dentro de la caja verde: una a la izquierda, otra a la derecha.
- 41,5 % + 56,5 % = 98 % → 2 % de separación.
- **Importante:** estos porcentajes se calculan sobre el **interior** de `.contenido` (su ancho menos el padding), no sobre la pantalla. Un porcentaje siempre se refiere al contenedor padre.

- `height: 279px`: 310 de la caja verde − 15 de padding arriba − 15 abajo = 280 ≈ 279.
- `border-top` y `border-left`: se pueden poner bordes solo en algunos lados.

```
.texto-central h2 {
  font-size: 14px;
  margin: 16px 0 14px;
}
```

Tamaño del subtítulo y su separación: 16px arriba, 0 a los lados, 14px abajo (hasta el párrafo).

### 5.6. Pie de página

```
.pie {
  height: 134px;
  background-color: pink;
  border-radius: 5px;
}
```

Altura fija por el mismo motivo de siempre: sus hijos flotan.

```
.caja {
  background-color: blue;
  color: white;
  text-align: center;
  border: 1px solid #333;
  border-radius: 5px;
}
```

Estilo **compartido** por las tres cajas (gracias a la clase `caja` que llevan todas).

```
.caja-direccion {
  float: left;
  width: 10%;
  height: 108px;
  margin: 15px 0 0 2%;
}

.caja-nombre {
  float: left;
  width: 25%;
  height: 40px;
  margin: 19px 0 0 23%;
}

.caja-imagenes {
  float: right;
  width: 25%;
  height: 107px;
  margin-top: 18px;
}
```

Cada caja tiene su tamaño y su posición, conseguida con floats y márgenes:

```
|2%| dirección 10% |  23%  | nombre 25% |  15% hueco  | imágenes 25% |
 ←──────────────── float: left ──────────────→             ← float: right
```

- La primera y la segunda flotan a la izquierda, una tras otra. El `margin-left` de cada una es el hueco que la separa de lo anterior.
- La tercera flota a la derecha, así que se pega al borde derecho sin hacer cuentas.
- Los `margin-top` distintos (15, 19, 18px) bajan cada caja un poco, como en la maqueta.
- Los márgenes en `%` (2 %, 23 %) también se calculan sobre el ancho del padre, así que se adaptan a la pantalla.

## 6. Resumen: la receta para maquetar sin flex ni grid

1. **Reset** con `* { margin: 0; padding: 0; box-sizing: border-box; }`.
2. Estructura el HTML en **bloques** (header, nav, contenedor, footer) que van uno debajo de otro.
3. Para poner cajas **en fila** → `float: left` (y `float: right` para la última si quieres que se pegue a la derecha).
4. Usa **anchos en %** y comprueba que **sumen 100 % o menos**. Lo que sobra queda como separación.
5. Si un contenedor solo tiene hijos flotantes, dale **altura** (o `overflow: hidden`) para que no se colapse.
6. Para colocar algo en un punto exacto dentro de una caja → padre con `position: relative` + hijo con `position: absolute` y `top/left/right/bottom`.
7. Para centrar texto: horizontal con `text-align: center`; vertical (una línea) con `line-height` igual a la altura.
8. Reutiliza estilos con **clases múltiples** (`class="caja caja-nombre"`).

## 7. Ejercicios para practicarlo tú

- Cambia `.lateral` a `width: 20%` y `.contenido` a `width: 78%`. ¿Qué pasa con el hueco? ¿Y si pones 40 % y 67 %? (Verás que la caja verde baja, porque no cabe.)
- Quita `height: 310px` de `.cuerpo` y mira cómo el pie "se cuela" debajo. Luego arréglalo con `overflow: hidden`.
- Cambia el `position: relative` de `.cabecera` por `static` y observa dónde aparece el botón "Buscar".
- Pon `position: sticky; top: 0; background: white;` a `.menu` y añade mucho texto al párrafo para poder hacer scroll: el menú se quedará pegado arriba.

- Quita `box-sizing: border-box` del reset y observa cómo se descolocan las cajas.